

Ciência da Computação | Pensamento Computacional

# A Arquitetura da Resiliência

Tratamento de Erros, Sistematização de Testes  
e Avaliação de Soluções

Protocolo de  
Qualidade de Software

**Criar um sistema inteligente é,  
acima de tudo, criar um sistema  
preparado para errar.**

— e saber reagir aos erros de forma segura, lógica e estruturada.

O objetivo não é apenas encontrar se existe erro, mas também qual é o erro e onde ele está.  
Alan Turing já questionava: Você está preparado?

## ERRO (A Origem)



Falha humana ocorrida em qualquer fase do desenvolvimento (planejamento, lógica ou digitação).



## BUG (O Sintoma)



A manifestação do erro humano dentro do software em execução.

**Regra de Ouro:** Quanto mais cedo o erro humano é descoberto, menor a chance de ele se transformar em um bug letal no sistema.

Erro Lógico por Ambiguidade

## Conflito de Regras

### Conflito de Regras

A falha na interpretação de requisitos.

Sistema de login que bloqueia após  
3 tentativas ou 5 tentativas?

Inconsistência estrutural. O código funciona, mas a regra de negócio colide.

Erro Lógico-Matemático

## Falha de Prioridade Operacional

### Falha de Prioridade Operacional

A falha na definição de condições ou precedência matemática.

```
preco = 100  
desconto = 10  
preco_final = preco - desconto / 100
```

⚠ Erro de Parênteses: O desconto é dividido antes da subtração. O sistema não quebra, mas o resultado é matematicamente falso.



## Simplicidade na Solução

Ferragina e Luccio (2018)

Soluções simples e diretas geram inerentemente menos espaço para erros ocultos. Complexidade desnecessária é o habitat natural dos bugs.



## Ação Defensiva

Prever falhas e preparar estados seguros.

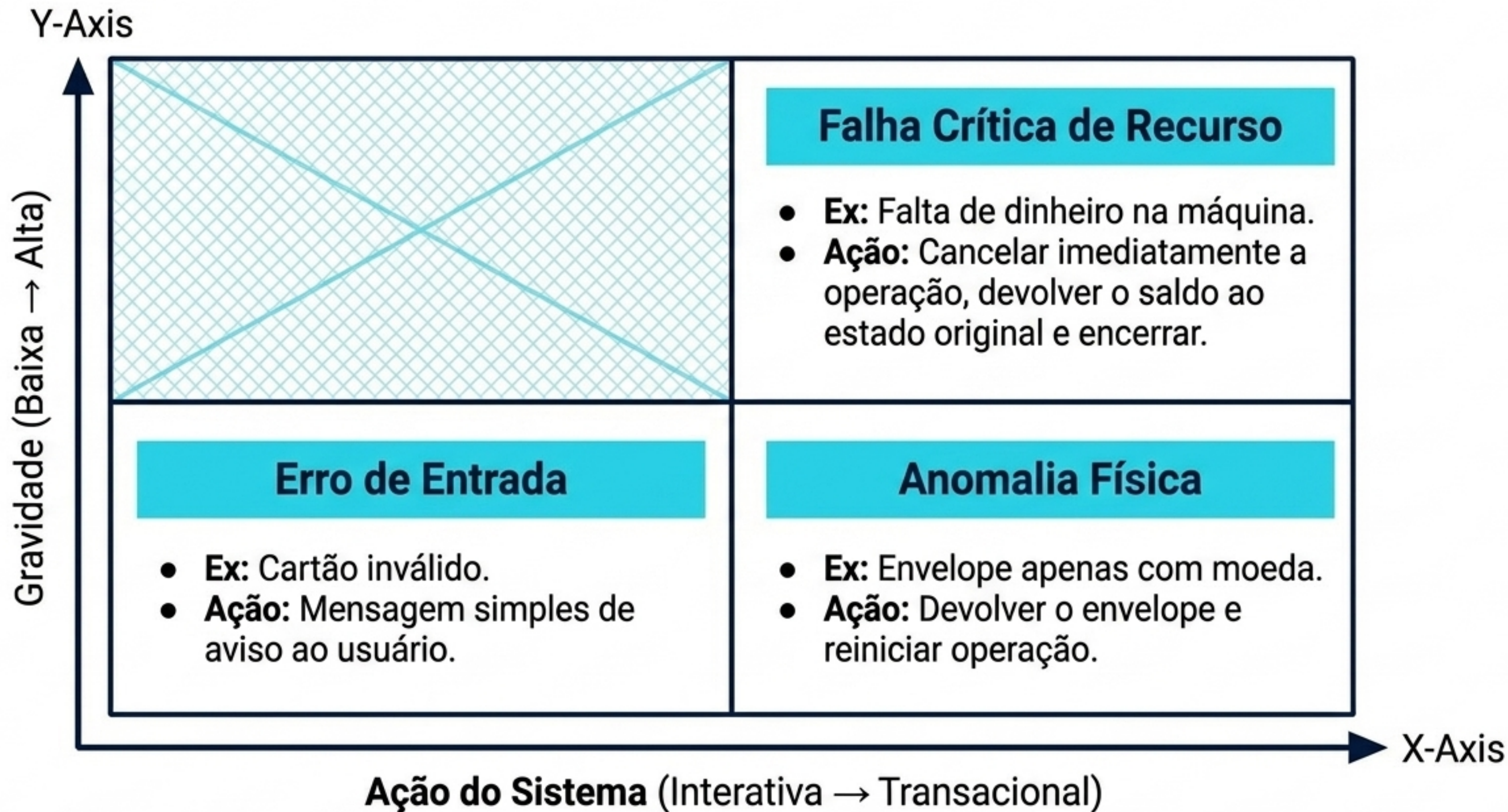
Divisão por zero? Crie uma pré-condição para impedir. Preparar o sistema para perda de conexão ou arquivos corrompidos.



## Planejamento (Pseudocódigo)

Riley e Hunt (2014)

Desenhar o comportamento antes de codificar. Prevenir erros estruturais ainda na fase de projeto e abstração.



**Sistemas interativos perguntam; sistemas transacionais desfazem.**

**Usuário**

**Sistema**

**Erros de Entrada  
(Digitação)**

Exemplo: Confundir  
a letra 'O' com o  
número '0'.

**Filtro de  
Validação**

Verificar quantidade  
de caracteres e tipo  
de dados (número,  
(número, letra,  
especial) antes de  
processar.

**Erros de Previsão**

Falhas na lógica  
interna ao tentar  
processar dados não  
higienizados.

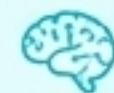


|                | Característica                                     | Vantagens                                                         | Ponto Cego                                                     |
|----------------|----------------------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------------|
| Top-down       | Testa o sistema como um todo.                      | Encontra falhas de projeto e arquitetura.                         | Difícil localizar erros específicos; testa sistema incompleto. |
| Bottom-up      | Testa partes pequenas isoladas (módulos, funções). | Fácil de aplicar cedo; cirúrgico para localizar problemas locais. | Não detecta falhas de integração no início.                    |
| Teste Estático | Analisa o código-fonte sem executá-lo.             | Detecta erros de sintaxe, estilo e variáveis ociosas.             | Cego para o comportamento real do sistema em execução.         |



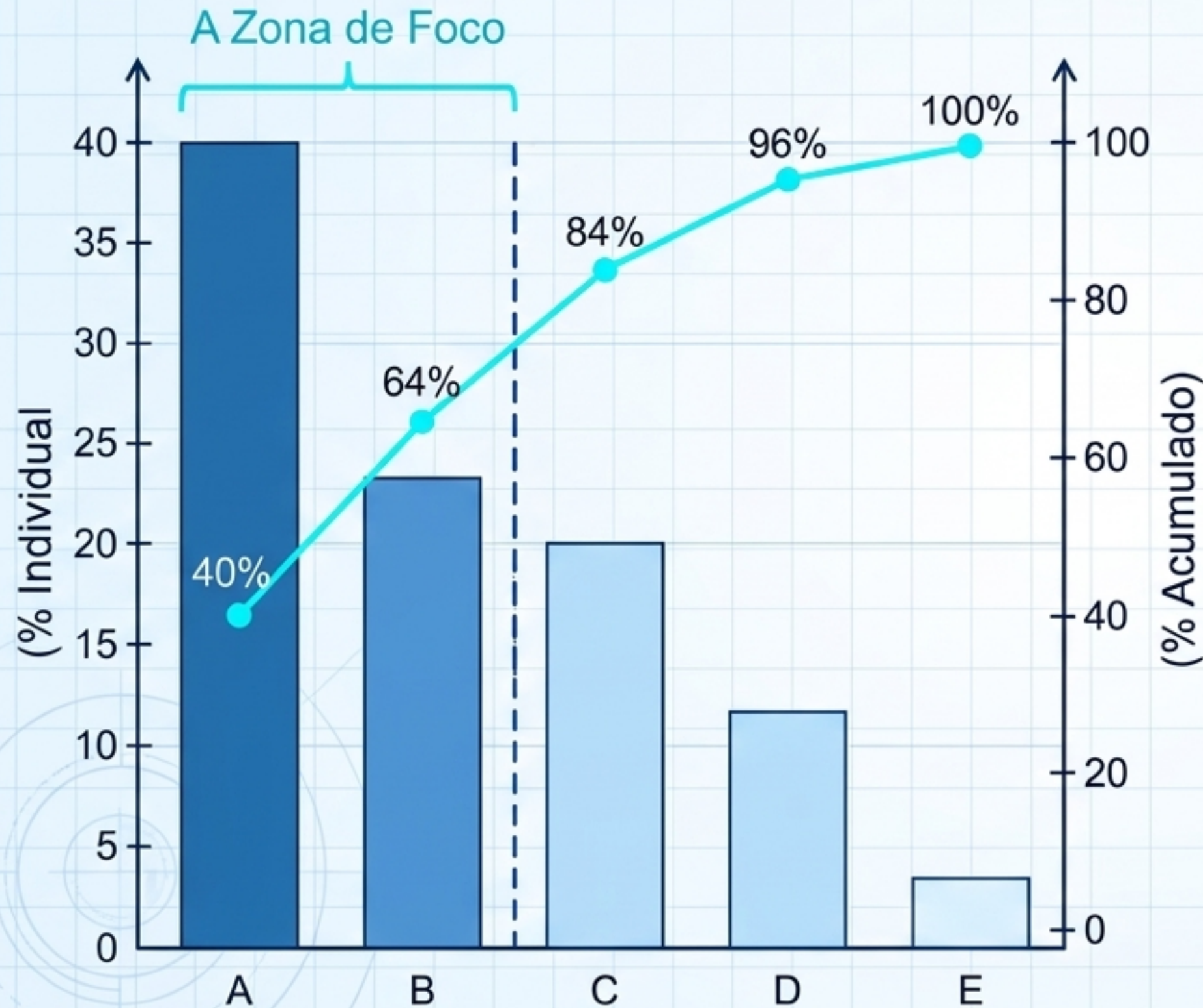
# A Filosofia do Teste de Unidade

- Testa **partes isoladas** do sistema (funções e métodos específicos).
- **Exige a simulação de componentes ausentes** (já que o ecossistema completo ainda não existe).



**Mensagem Central:** Testar de forma sistemática é como organizar a bagunça antes dela virar um problema maior: primeiro testamos partes pequenas, depois vemos o todo. Um sistema de qualidade é construído desde a primeira linha.

# Gráfico de Pareto – Tipos de Falha em Software



## Priorização Estratégica (Pareto)

Em vez de tentar corrigir todos os erros simultaneamente, o desenvolvedor foca onde o impacto é máximo.

### Diagnostic Questions

- Onde ocorrem mais falhas?
- Quais erros causam mais interrupções críticas?
- Quais partes do sistema concentram os bugs mais letais?

## PREVENÇÃO

Planejamento,  
Pseudocódigo,  
Lógica Defensiva.

## SISTEMATIZAÇÃO

Modelo em U, Testes  
Top-down / Bottom-up.

**A excelência não é escrever código  
sem erro. É orquestrar um ambiente  
onde o erro é previsto, isolado e  
neutralizado antes de causar danos.**

## DEPURAÇÃO

Gráfico de Pareto,  
Rastreamento passo a passo.

# Missão Tática: "Caça aos Bugs"

*Laboratório Prático de Tratamento de Erros e Avaliação de Soluções (Prof. Kadidja Valéria / Baseado em Santos, 2021).*

## Os 3 Objetivos da Operação

### 1. Identificar

Localizar erros ocultos (sintaxe, lógica ou execução) em um algoritmo imperfeito.

### 2. Corrigir

Refatorar o código e documentar a justificativa técnica para cada alteração.

### 3. Avaliar

Auditar a solução final com base em critérios de clareza, eficiência e escalabilidade.

# Aula\_Erros\_Debugging\_GrupoX

- 📄 **README.md** → Explicação da atividade e objetivos.
- 📄 **Algoritmo\_Erros.md** → A versão original, com mapeamento dos bugs encontrados.
- 📄 **Algoritmo\_Corrigido.md** → O código limpo, comentado com as justificativas da refatoração.
- 📄 **Avaliacao.md** → Reflexão crítica sobre a eficiência da nova solução.

```
> git commit -m "Entrega Aula - Tratamento de Erros e Avaliação de Soluções"
```

“

**“Um programador de verdade não é aquele que escreve códigos sem errar. É aquele que organiza seu pensamento para prever, testar, corrigir e melhorar seus sistemas antes que os erros causem danos.”**